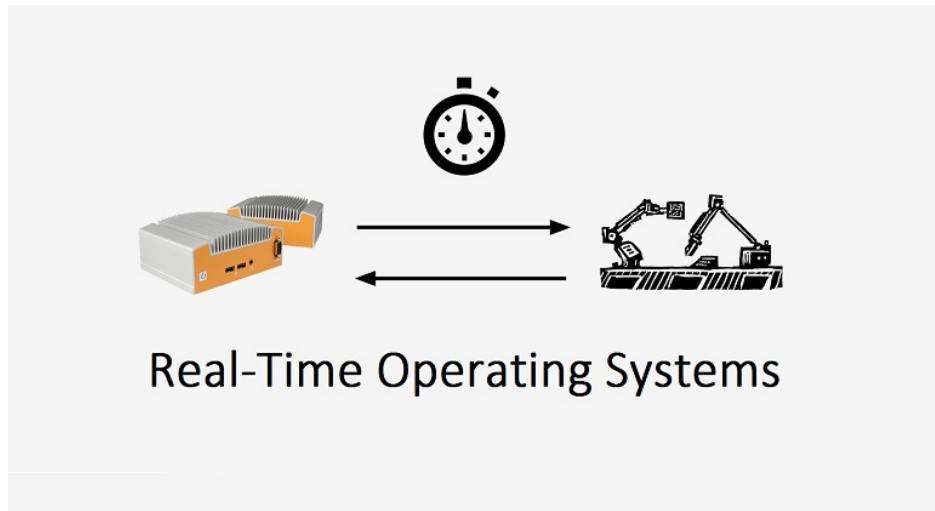




Real-time Operating Systems

LAB MODULE I - RT SERVICES ON LINUX
2026/2027

Paulo Pedreiras
DETI/UA/IT
pbrp@ua.pt

September 10, 2026

1 Objectives

- Learn how to use the basic Linux real-time services, including creating periodic threads, assigning real-time priorities to threads, handling shared resources in real-time, etc.
- Use programmatic methods and OS tools to evaluate the real-time performance of threads
- Develop a simple real-time application in Linux.

2 Procedure

The Linux RT services laboratory activity comprises two parts.

The first one has a tutorial nature, aiming to allow students to get familiar with the basic methodologies and tools used for developing real-time applications in Linux. Its realization is optional, but highly recommended, not being subject to formal assessment.

The second part consists of the development of a real-time application in Linux, where students are expected to use the real-time model and the tools available in Linux to obtain the best possible real-time performance.

3 Part I: Basic methodologies and tools

Below follows a set of tasks that students should carry out in order to get familiar with the basic methods and tools available in Linux to develop real-time applications.

- **[PI-01]**: Download the sample code provided at the eLearning course page. Execute the code with and without load and analyze the output
- **[PI-02]**: Modify the code in **[PI-01]** to allow passing two command line arguments:
 - i) priority (integer, [1,99]) and,
 - ii) periodicity (integer, *ms* resolution, range [50,500] *ms*).Execute the code with and without load, with different priority levels, and analyze output.

- **[PI-03]**: Execute several instances of the code developed in **[PI-02]**, in different terminals, with different priorities and periodicity. Periods should not be harmonic and should generate different phase conditions at runtime (e.g. use {90 ms, 95 ms, 107 ms, ...}). Analyze the results.
- **[PI-04]**: Modify the code developed in **[PI-03]** so that processes are executed in the same CPU. Repeat the procedure, i.e. execute several instances of the code in different terminals, with different priorities and periodicity. Periods should not be harmonic, generating different phase conditions at runtime (e.g. use {90 ms, 95 ms, 107 ms, ...}). Analyze the results and compare them with the previous ones.
- **[PI-05]**: Many real-time applications involve acquiring signals from the environment (lato sensu, e.g. data obtained from sensors, packets that arrive via a network interface in the case of distributed architectures), followed by some sort of pre-processing before the data is actually used (for registration, for visualization, for control, for a combination of these, etc.)

According to the real-time model, activities are split in tasks that carry out one specific function (e.g. data acquisition, filtering, display, etc.) and communicate and synchronize via suitable mechanisms. In this exercise you are challenged to create an application according to the real-time model. The focus is on how to structure an application, not on the processing itself, so the functionality is quite basic.

More specifically, the proposed challenge is to create an application composed of three real-time threads/tasks. The first task emulates a temperature sensor. To avoid the need of having a real sensor and also for testing/debugging purposes (**why?**), the sensor samples are statically defined, using an array (assume that samples are of `uint16_t` type). This task is periodic and is activated once every second. Each “sample” is placed in one FIFO queue (supported on an array - static memory) with capacity of 2 samples (**is this a good choice? Why?**). A second task reads the temperature samples and applies a filter. To keep it simple, assume a basic moving average filter with 4 points. The filtered data (still of `uint16_t`) should be placed in another FIFO queue (also supported on an array - static memory) with capacity of 4 samples (**again, is this a good choice? Why?**). Finally, a third task is responsible to show the data on the display.

As mentioned above, the sensor task is periodic, with a period of 1 second. The filter and the display tasks are sporadic and its execution order should follow the “data flow” – acquisition, filter and display. The generator thread should have higher priority than the other tasks, followed by the filter task (**Is this reasonable? Why?**). Synchronization and mutual exclusion should be imposed by means of mutexes and condition variables.

4 Part II: Development of a real-time application

To be announced